KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Department of Computer Science & Engineering — A.Y 2026-2027

Software Engineering Lab Internal - I

Complete Step-by-Step Lab Guides for All 9 Sets

Includes: pom.xml (Original + Corrected) for All Sets

Subject Code: 23CC505PC / 23CC501PC | Year & Semester: III / I | Branch: CSE

Covers: Maven | Git & GitHub | Docker

Software Engineering Lab Exam - Step-by-Step Lab Guides (All 9 Sets)

APPENDIX: pom.xml FILES FOR ALL 9 SETS

(Original with Errors + Corrected Ready-to-Paste)

SET-1: Hospital Management System (HMS)

Repository: https://github.com/ssvkotamraju/HMS.git

Original pom.xml (with errors)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.library</groupId>
    <artifactId>LibraryRegistration</artifactId>

    <dependencies>
        <!-- JSP API -->
        <dependency>
            <groupId>javax.servlet.jsp</groupId>
            <artifactId>javax.servlet.jsp-api</artifactId>
            <version>2.3.3</version>
            <scope>provided</scope>
        </dependency>

       <build>
        <finalName>LibraryRegistration</finalName>
    </build>
</project>
```

Errors Identified

- **Missing** `<version>` — No project version specified
- **Missing** `<packaging>war</packaging>` — Required for web applications
- `<dependencies>` **tag not closed** — The `</dependencies>` closing tag is missing before `<build>`
- **Missing Servlet API dependency** — Only JSP API is included; the Servlet API (`javax.servlet-api`) is also needed

Corrected pom.xml (ready to paste)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.library</groupId>
    <artifactId>LibraryRegistration</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>
```

```
    <dependencies>
        <!-- Servlet API -->
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>4.0.1</version>
            <scope>provided</scope>
        </dependency>
        <!-- JSP API -->
        <dependency>
            <groupId>javax.servlet.jsp</groupId>
            <artifactId>javax.servlet.jsp-api</artifactId>
            <version>2.3.3</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <finalName>LibraryRegistration</finalName>
    </build>
</project>
```

SET-2: E-Ticketing System (OTS)

Repository: https://github.com/ssvkotamraju/E-ticketing.git

Original pom.xml (with errors)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.library</groupId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>

    <dependencies>
        <!-- JSP API -->
        <dependency>
            <groupId>javax.servlet.jsp</groupId>
            <artifactId>javax.servlet.jsp-api</artifactId>
            <version>2.3.3</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <finalName>LibraryRegistration</finalName>
    </build>
</project>
```

Errors Identified

- **Missing** `<artifactId>` — Every Maven project must have a unique artifactId
- **Missing Servlet API dependency** — Required for any web application with JSP/Servlets
- `<finalName>` **mismatch** — Should reflect the E-Ticketing project, not "LibraryRegistration"

Corrected pom.xml (ready to paste)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.library</groupId>
    <artifactId>E-ticketing</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>

    <dependencies>
        <!-- Servlet API -->
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>4.0.1</version>
            <scope>provided</scope>
        </dependency>
        <!-- JSP API -->
        <dependency>
            <groupId>javax.servlet.jsp</groupId>
            <artifactId>javax.servlet.jsp-api</artifactId>
            <version>2.3.3</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <finalName>E-ticketing</finalName>
    </build>
</project>
```

SET-3: Online Recruitment System (ORS)

Repository: https://github.com/ssvkotamraju/ORS.git

Original pom.xml (with errors)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT</groupId>
  <artifactId>ORS</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>ORS Webapp</name>
  <url>http://www.example.com</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>
  <dependencies>
  </dependencies>
  <build>
    <finalName>ORS</finalName>
    <pluginManagement>
```

```
      <plugins>
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-compiler-plugn</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-jre-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

Errors Identified

- **Missing** `<packaging>war</packaging>` — Defaults to jar without it; web apps need war
- **Typo:** `maven-compiler-plugn` — Missing letter 'i'; should be `maven-compiler-plugin`
- **Wrong plugin:** `maven-jre-plugin` — No such plugin exists; should be `maven-war-plugin`
- **Empty** `<dependencies>` **section** — No servlet/JSP dependencies for a web application

Corrected pom.xml (ready to paste)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT</groupId>
  <artifactId>ORS</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>ORS Webapp</name>
  <url>http://www.example.com</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
```

```
    <dependency>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>javax.servlet.jsp-api</artifactId>
      <version>2.3.3</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>ORS</finalName>
    <pluginManagement>
      <plugins>
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-war-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

SET-4: Online Shopping Application

Repository: https://github.com/nikithamoturi/OnlineShoppingApp.git

Original pom.xml (with errors)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT.org</groupId>
  <artifactId>OnlineShoppingApp</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>OnlineShoppingApp Maven Webapp</name>
  <url>http://www.example.com</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
```

```
  </properties>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <!-- ... pluginManagement omitted for brevity ... -->
</project>
```

Errors Identified

- **Missing Servlet API dependency** — Required for JSP/Servlet web application
- **Missing JSP API dependency** — Required for compiling JSP pages
- Only JUnit is declared; no web dependencies at all

Corrected pom.xml (ready to paste)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT.org</groupId>
  <artifactId>OnlineShoppingApp</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>OnlineShoppingApp Maven Webapp</name>
  <url>http://www.example.com</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>javax.servlet.jsp-api</artifactId>
      <version>2.3.3</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>OnlineShoppingApp</finalName>
    <pluginManagement>
      <plugins>
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
```

```
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-resources-plugin</artifactId>
          <version>3.3.1</version>
        </plugin>
        <plugin>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-war-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

SET-5: Blood Bank Management System

Repository: https://github.com/sarasrija/Blood-Bank-Management-system.git

Original pom.xml (with errors)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.bloodbank</groupId>
  <artifactId>BloodBankManagementSystem</artifactId>
  <packaging>war</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>Blood Bank Management System</name>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided
    </dependency>
  <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-invalid-artifact</artifactId>
      <version>99.99.99</version>
    </dependency>
       <dependency>
      <groupId>junit</groupId>
```

```
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
    </dependency>
  </dependencies>
  <build>
    <finalName>BloodBankManagementSystem</finalName>
    <plugins>
      <plugin>
        <artifactId>tomcat7-maven-plugin</artifactId>
        <version>2.2</version>
        <configuration>
          <path>/</path>
          <port>8080</port>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Errors Identified

- `<scope>provided` **not closed** — Missing `</scope>` closing tag
- **MySQL groupId wrong** — `com.mysql` with invalid artifactId `mysql-connector-invalid-artifact`; correct is `com.mysql` with `mysql-connector-j`
- **MySQL version** `99.99.99` **doesn't exist** — Use a real version like `8.0.33`
- **JUnit missing** `<scope>test</scope>` — Should be scoped to test
- **Tomcat7 plugin missing** `<groupId>` — Needs `<groupId>org.apache.tomcat.maven</groupId>`

Corrected pom.xml (ready to paste)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.bloodbank</groupId>
  <artifactId>BloodBankManagementSystem</artifactId>
  <packaging>war</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>Blood Bank Management System</name>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.0.33</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>BloodBankManagementSystem</finalName>
```

```
    <plugins>
      <plugin>
        <groupId>org.apache.tomcat.maven</groupId>
        <artifactId>tomcat7-maven-plugin</artifactId>
        <version>2.2</version>
        <configuration>
          <path>/</path>
          <port>8080</port>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

SET-6: Sports Management System

Repository: https://github.com/sarasrija/SportManagementSystem.git

Original pom.xml (with errors)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>SELABINTERNAL1</groupId>
  <artifactId>SportsTournamentManagementSystem</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>StudentCourseManagementSystem Maven Webapp</name>
  ...
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artificatId>javax.servlet-api</artificatId>   <!-- TYPO -->
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>mysql-db</groupId>                    <!-- WRONG -->
      <artifactId>mysql-connector</artifactId>        <!-- WRONG -->
      <version>8.0.33</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <scope>test</scope>                             <!-- MISSING VERSION -->
    </dependency>
    <dependency>
      <groupId>com.sports</groupId>
      <artifactId>sports-analytics</artifactId>       <!-- VENDOR LIB -->
      <version>1.0.0</version>
    </dependency>
  </dependencies>
  <build>
    <finalName>StudentCourseManagementSystem</finalName>
    ...
    <plugins>
      <plugin>
        <artifactId>tomcat7-maven-plugin</artifactId>  <!-- MISSING groupId -->
        <version>2.2</version>
        ...
```

```
      </plugin>
    </plugins>
  </build>
</project>
```

Errors Identified

- **Typo:** `<artificatId>` — Should be `<artifactId>` (appears twice: opening and closing)
- **MySQL** `groupId` **wrong** — `mysql-db` should be `com.mysql`
- **MySQL** `artifactId` **wrong** — `mysql-connector` should be `mysql-connector-j`
- **JUnit missing** `<version>` — Every dependency must specify a version
- **Tomcat7 plugin missing** `<groupId>` — Needs `org.apache.tomcat.maven`
- `sports-analytics` **is a vendor library** — Not in Maven Central; must be installed locally via `mvn install:install-file`

Corrected pom.xml (ready to paste)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>SELABINTERNAL1</groupId>
  <artifactId>SportsTournamentManagementSystem</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>SportsTournamentManagementSystem Maven Webapp</name>
  <url>http://www.example.com</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.0.33</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>com.sports</groupId>
      <artifactId>sports-analytics</artifactId>
      <version>1.0.0</version>
    </dependency>
  </dependencies>
  <build>
    <finalName>SportsTournamentManagementSystem</finalName>
    <pluginManagement>
      <plugins>
        <plugin>
```

```
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-war-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
      </plugins>
    </pluginManagement>
    <plugins>
      <plugin>
        <groupId>org.apache.tomcat.maven</groupId>
        <artifactId>tomcat7-maven-plugin</artifactId>
        <version>2.2</version>
        <configuration>
          <port>8080</port>
          <path>/SportsTournament</path>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Note: For the vendor library `sports-analytics`, install it locally:

```
mvn install:install-file -Dfile=sports-analytics.jar -DgroupId=com.sports -DartifactId=sports-analytics -
Dversion=1.0.0 -Dpackaging=jar
```

SET-7: Apartment Management System

Repository: https://github.com/sarasrija/ApartmentManagementSystem.git

Original pom.xml (with errors)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.apartment</groupId>
  <artifactId>ApartmentMaintenanceSystem</artifactId>
  <packaging>war</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>Apartment Maintenance System</name>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided           <!-- MISSING </scope> -->
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-invalid-artifact</artifactId>
      <version>99.99.99</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <!-- Missing <scope>test</scope> -->
```

```
    </dependency>
  </dependencies>
  <build>
    <finalName>ApartmentMaintenanceSystem</finalName>
    <plugins>
      <plugin>
        <artifactId>tomcat7-maven-plugin</artifactId>  <!-- MISSING groupId -->
        ...
      </plugin>
    </plugins>
  </build>
</project>
```

Errors Identified

- `<scope>provided` **not closed** — Missing `</scope>` closing tag
- **MySQL** `artifactId` **wrong** — `mysql-connector-invalid-artifact` should be `mysql-connector-j`
- **MySQL** `version` **doesn't exist** — `99.99.99` is invalid; use `8.0.33`
- **JUnit missing** `<scope>test</scope>`
- **Tomcat7 plugin missing** `<groupId>org.apache.tomcat.maven</groupId>`

Corrected pom.xml (ready to paste)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.apartment</groupId>
  <artifactId>ApartmentMaintenanceSystem</artifactId>
  <packaging>war</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>Apartment Maintenance System</name>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.0.33</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>ApartmentMaintenanceSystem</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.tomcat.maven</groupId>
        <artifactId>tomcat7-maven-plugin</artifactId>
        <version>2.2</version>
        <configuration>
```

```
          <path>/</path>
          <port>8080</port>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

SET-8: Vehicle Rental Management System

Repository: https://github.com/nikithamoturi/VehicleRentalManagementSystem.git

Original pom.xml (with errors)

```
<?xml version="1.0" encoding="UTF-8"?>
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT</groupId>
  <artifactId>VehicleRentalManagement</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  ...
  <dependencies>
  </dependencies>
  <!-- ... pluginManagement with standard plugins ... -->
</project>
```

Errors Identified

- **Empty `<dependencies>` section** — Missing servlet-api and JSP API dependencies required for a web application

Corrected pom.xml (ready to paste)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT</groupId>
  <artifactId>VehicleRentalManagement</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>VehicleRentalManagement Maven Webapp</name>
  <url>http://www.example.com</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet.jsp</groupId>
```

```
      <artifactId>javax.servlet.jsp-api</artifactId>
      <version>2.3.3</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>VehicleRentalManagement</finalName>
    <pluginManagement>
      <plugins>
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-resources-plugin</artifactId>
          <version>3.3.1</version>
        </plugin>
        <plugin>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-war-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

SET-9: Gym Membership Management System

Repository: https://github.com/nikithamoturi/GymManagementSystem.git

Original pom.xml (with errors)

```
<?xml version="1.0" encoding="UTF-8"?>
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT</groupId>
  <artifactId>GymManagementSystem</artifactId>
  <version>0.0.1-SNAPSHOT</version>
```

```
  <packaging>war</packaging>
  ...
  <dependencies>
  </dependencies>
  <!-- ... pluginManagement with standard plugins ... -->
</project>
```

Errors Identified

- **Empty `<dependencies>` section** — Missing servlet-api and JSP API dependencies required for a web application

Corrected pom.xml (ready to paste)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>KMIT</groupId>
  <artifactId>GymManagementSystem</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>GymManagementSystem Maven Webapp</name>
  <url>http://www.example.com</url>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>javax.servlet.jsp-api</artifactId>
      <version>2.3.3</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>GymManagementSystem</finalName>
    <pluginManagement>
      <plugins>
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-resources-plugin</artifactId>
          <version>3.3.1</version>
        </plugin>
        <plugin>
          <artifactId>maven-compiler-plugin</artifactId>
```

```
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-war-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

PART B: DETAILED EXAM LAB GUIDES

I'll structure this exactly in the order you should perform it in the lab, using the applications you said you have: Eclipse, Maven, Git Bash, Docker, Internet/GitHub access.

SET-1

PART 1 — Clone and Build the Hospital Management System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir HMS-Project
```

```
cd HMS-Project
```

Now clone the repository given in the question:

```
git clone https://github.com/ssvkotamraju/HMS.git
```

```
cd HMS
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins. Fix any missing dependencies or library conflicts mentioned in the error logs.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Initialize Git and create the first commit

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

Step 14: Mobile number & pwd validation workflow

```
git checkout -b feature/registration-enhancement
```

make changes to the files

```
git add .
```

```
git commit -m "Add mobile number and improved password validation"
```

Step 15: Inspect changes before commit

```
git diff
```

Step 16: Prepare untracked/modified changes for a commit

```
git add register.jsp new-style.css
```

Step 17: Undo commit preserving history

```
git revert HEAD
```

Step 18: Remove file from staging area without deleting from disk

```
git restore --staged config.txt
```

or: git rm --cached config.txt

Step 19: Stash changes to fix urgent issue

```
git stash
```

```
git checkout main
```

fix issue and commit

```
git checkout feature/registration-enhancement
```

```
git stash pop
```

Step 20: Clean project history via rebase

```
git checkout feature/registration-enhancement
```

```
git rebase main
```

Step 21: Recover an important commit

```
git reflog
```

```
git cherry-pick <commit-hash>
```

Step 22: Configure remote and push changes

```
git remote add origin https://github.com/ssvkotamraju/HMS.git
```

```
git push -u origin main
```

PART 3 — DOCKER Step 23: First find your WAR/JAR Look inside `target/` for your generated `.war` or `.jar` file.

Step 24: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM tomcat:9-jdk17
```

```
COPY target/HMS.war /usr/local/tomcat/webapps/ROOT.war
```

```
EXPOSE 8080
```

```
CMD ["catalina.sh", "run"]
```

Step 25: Build Docker image

```
docker build -t HMS .
```

Step 26: Run the container

```
docker run -d -p 8080:8080 --name hms-container HMS
```

Step 27: Check the container

```
docker ps
```

Open browser: http://localhost:8080

Step 28: Troubleshooting and Port Mapping If it fails, check logs: `docker logs hms-container` Check ports: `docker port hms-container`

Step 29: Useful Docker commands to memorize

```
docker images, docker build -t HMS ., docker ps, docker ps -a, docker logs, docker stop, docker rm.
```

```
docker tag HMS username/HMS
```

```
docker push username/HMS
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone [REPO_URL]
```

```
                ↓
```

```
cd [Project]
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
         dependencies
                ↓
      Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                    ↓
```

```
git branch
```

```
                    ↓
             Git operations
                    ↓
                Dockerfile
                    ↓
```

```
mvn clean install
```

```
                    ↓
```

```
docker build -t [IMAGE_NAME] .
```

```
                    ↓
```

```
docker run -d -p
```

```
          8080:8080 [IMAGE_NAME]
                    ↓
```

```
docker ps
```

```
                    ↓
```

```
http://localhost:8080
```

```
                    ↓
                  DONE
```

⭐ Most important commands to remember

```
mvn clean install
```

```
mvn -version
```

```
mvn dependency:tree
```

```
mvn package -DskipTests
```

```
git clone
```

```
git status
```

```
git add .
```

```
git commit -m "message"
```

```
git push
```

```
git rebase
```

```
git stash
```

```
docker build -t HMS .
```

```
docker run -d -p 8080:8080 HMS
```

```
docker ps
```

```
docker logs hms-container
```

SET-2

PART 1 — Clone and Build the E-Ticketing System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir OTS-Project
```

```
cd OTS-Project
```

Now clone the repository given in the question:

```
git clone https://github.com/ssvkotamraju/E-ticketing.git
```

```
cd E-ticketing
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins. Fix any missing dependencies or library conflicts mentioned in the error logs.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Create the first commit

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

Step 14: Create separate branch for search feature

```
git checkout -b feature/search-trains
```

Step 15: Review changes before committing

```
git diff
```

```
git diff --staged
```

Step 16: Discard uncommitted changes

```
git checkout -- train-search.html
```

or: git restore train-search.html

Step 17: Move branch back to an earlier commit

```
git reset --hard <commit-hash>
```

Step 18: Identify and delete unused branches

```
git branch --merged
```

```
git branch -d feature-old-branch
```

Step 19: Merge separate branches into main

```
git checkout main
```

```
git merge feature-train-search
```

```
git merge feature-passenger-registration
```

Step 20: Create, inspect, and apply Git patch

Create patch

```
git format-patch -1 HEAD
```

Inspect patch

```
git apply --stat patchfile.patch
```

Apply patch

```
git apply patchfile.patch
```

Step 21: Troubleshoot and resolve patch conflict

```
git am -3 patchfile.patch
```

resolve conflicts manually

```
git add .
```

```
git am --continue
```

Step 22: Configure remote and push changes

```
git remote add origin https://github.com/ssvkotamraju/E-ticketing.git
```

```
git push -u origin main
```

PART 3 — DOCKER Step 23: First find your WAR/JAR Look inside `target/` for your generated `.war` or `.jar` file.

Step 24: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM tomcat:9-jdk17
```

```
COPY target/E-ticketing.war /usr/local/tomcat/webapps/ROOT.war
```

```
EXPOSE 8080
```

```
CMD ["catalina.sh", "run"]
```

Step 25: Build Docker image

```
docker build -t OTS .
```

Step 26: Run the container

```
docker run -d -p 8080:8080 --name ots-container OTS
```

Step 27: Check the container

```
docker ps
```

Open browser: http://localhost:8080

Step 28: Troubleshooting and Port Mapping If it fails, check logs: `docker logs ots-container` Check ports: `docker port ots-container`

Step 29: Useful Docker commands to memorize

```
docker images, docker build -t OTS ., docker ps, docker ps -a, docker logs, docker stop, docker rm.
```

```
docker tag OTS username/OTS
```

```
docker push username/OTS
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone [REPO_URL]
```

```
                ↓
```

```
cd [Project]
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
         dependencies
                ↓
      Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                       ↓
```

```
git branch
```

```
                       ↓
              Git operations
                       ↓
                 Dockerfile
                       ↓
```

```
mvn clean install
```

```
                    ↓
```

```
docker build -t [IMAGE_NAME] .
```

```
                    ↓
```

```
docker run -d -p
```

```
          8080:8080 [IMAGE_NAME]
                    ↓
```

```
docker ps
```

```
                    ↓
```

```
http://localhost:8080
```

```
                    ↓
                  DONE
```

⭐ Most important commands to remember

```
mvn clean install
```

```
mvn -version
```

```
mvn dependency:tree
```

```
git checkout -b <branch>
```

```
git add .
```

```
git commit -m "message"
```

```
git push
```

```
git merge <branch>
```

```
git reset --hard
```

```
git apply patchfile
```

```
docker build -t OTS .
```

```
docker run -d -p 8080:8080 OTS
```

```
docker ps
```

```
docker logs ots-container
```

SET-3

PART 1 — Clone and Build the Online Recruitment System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir ORS-Project
```

```
cd ORS-Project
```

Now clone the repository given in the question:

```
git clone https://github.com/ssvkotamraju/ORS.git
```

```
cd ORS
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins. Fix any missing dependencies or library conflicts mentioned in the error logs.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Clone, create working branch, verify remote

Assuming cloned already, if not:

git clone https://github.com/ssvkotamraju/ORS.git

```
git checkout -b feature/job-application
```

```
git remote -v
```

Step 14: Retrieve remote changes and check status

```
git fetch origin
```

```
git status
```

Step 15: Update local branch and rebase

```
git fetch origin
```

```
git rebase origin/main
```

Step 16: Handle rebase conflict

Edit file in IDE to resolve conflict

```
git add apply.jsp
```

```
git rebase --continue
```

Step 17: Systematically resolve all conflicts

Follow Step 16 repeatedly until:

```
git status # shows rebase is successful
```

Step 18: Restore missing changes via reflog

```
git reflog
```

```
git cherry-pick <commit-hash>
```

Step 19: Cancel the rebase safely

```
git rebase --abort
```

Step 20: Configure SSH key and retry push ssh-keygen -t ed25519 -C "your_email@example.com"

Add the key to GitHub via UI from ~/.ssh/id_ed25519.pub

ssh -T git@github.com

```
git remote set-url origin git@github.com:ssvkotamraju/ORS.git
```

```
git push -u origin feature/job-application
```

Step 21: Verify correct version and commits

```
git log
```

```
git diff origin/main
```

Step 22: Push after a rebase

```
git push --force-with-lease
```

PART 3 — DOCKER Step 23: First find your WAR/JAR Look inside `target/` for your generated `.war` or `.jar` file.

Step 24: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM tomcat:9-jdk17
```

```
COPY target/ORS.war /usr/local/tomcat/webapps/ROOT.war
```

```
EXPOSE 8080
```

```
CMD ["catalina.sh", "run"]
```

Step 25: Build Docker image

```
docker build -t ORS .
```

Step 26: Run the container

```
docker run -d -p 8080:8080 --name ors-container ORS
```

Step 27: Check the container

```
docker ps
```

Open browser: http://localhost:8080

Step 28: Troubleshooting and Port Mapping If it fails, check logs: `docker logs ors-container` Check ports: `docker port ors-container`

Step 29: Useful Docker commands to memorize

```
docker images, docker build -t ORS ., docker ps, docker ps -a, docker logs, docker stop, docker rm.
```

```
docker tag ORS username/ORS
```

```
docker push username/ORS
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone [REPO_URL]
```

```
                ↓
```

```
cd [Project]
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
       dependencies
                ↓
      Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                    ↓
```

```
git branch
```

```
                    ↓
             Git operations
                    ↓
                Dockerfile
                    ↓
```

```
mvn clean install
```

```
                    ↓
```

```
docker build -t [IMAGE_NAME] .
```

```
                    ↓
```

```
docker run -d -p
```

```
          8080:8080 [IMAGE_NAME]
                    ↓
```

```
docker ps
```

```
                    ↓
```

```
http://localhost:8080
```

```
                    ↓
                  DONE
```

⭐ Most important commands to remember

```
mvn clean install
```

```
mvn -version
```

```
mvn dependency:tree
```

```
git remote -v
```

```
git fetch origin
```

```
git rebase origin/main
```

```
git rebase --continue
```

```
git rebase --abort
```

```
git push --force-with-lease
```

ssh-keygen -t ed25519

```
docker build -t ORS .
```

```
docker run -d -p 8080:8080 ORS
```

```
docker ps
```

```
docker logs ors-container
```

SET-4

PART 1 — Clone and Build the Online Shopping Application Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir OnlineShoppingLab
```

```
cd OnlineShoppingLab
```

Now clone the repository given in the question:

```
git clone https://github.com/nikithamoturi/OnlineShoppingApp.git
```

```
cd OnlineShoppingApp
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins. Fix any missing dependencies or compilation errors if required by updating versions in pom.xml to match your installed Java version.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Verify Repository Status and Remote (Q3a)

```
git status
```

```
git remote -v
```

Step 14: Create Feature Branch (Q3b)

```
git checkout -b feature/product-update
```

```
git branch
```

Step 15: Modify JSP and View Changes (Q3c) (Modify products.jsp in Eclipse to add Category and Quantity)

```
git diff
```

Step 16: Stage, Commit, and Verify (Q3d)

```
git add products.jsp
```

```
git commit -m "Update products.jsp with Category and Quantity"
```

```
git log -1
```

Step 17: Additional Modification and Diff (Q4a) (Make a small modification to products.jsp)

```
git diff
```

Step 18: Retrieve Remote Changes (Q4b)

```
git fetch origin
```

```
git rebase origin/main
```

Step 19: Compare Branch with Main (Q4c)

```
git diff main..feature/product-update
```

Step 20: Merge to Main and Push (Q4d)

```
git checkout main
```

```
git merge feature/product-update
```

```
git push origin main
```

PART 3 — DOCKER Step 21: First find your WAR/JAR Look inside `target/` for your generated `.war` or `.jar` file.

Step 22: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM tomcat:9-jdk17
```

```
COPY target/*.war /usr/local/tomcat/webapps/ROOT.war
```

```
EXPOSE 8080
```

```
CMD ["catalina.sh", "run"]
```

Step 23: Build Docker image

```
docker build -t OnlineShopping .
```

Step 24: Run the container

```
docker run -d -p 8080:8080 --name onlineshopping-container OnlineShopping
```

Step 25: Check the container

```
docker ps
```

Open browser: http://localhost:8080

Step 26: Troubleshooting and Port Mapping If it fails, check logs: `docker logs onlineshopping-container` Check ports: `docker port onlineshopping-container`

Step 27: Tag and Push Image to Docker Hub (Q6c, Q6d)

```
docker tag OnlineShopping your_username/OnlineShopping:v1
```

```
docker login
```

```
docker push your_username/OnlineShopping:v1
```

Step 28: Useful Docker commands to memorize

```
docker images, docker build -t [img] ., docker ps, docker ps -a, docker logs, docker stop, docker rm, docker tag, docker push.
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone https://github.com/nikithamoturi/OnlineShoppingApp.git
```

```
                ↓
```

```
cd OnlineShoppingApp
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
         dependencies
                ↓
      Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                       ↓
```

```
git branch
```

```
                       ↓
              Git operations
                       ↓
                 Dockerfile
                       ↓
```

```
mvn clean install
```

```
                       ↓
```

```
docker build -t OnlineShopping .
```

```
                    ↓
```

```
docker run -d -p
```

```
          8080:8080 OnlineShopping
                    ↓
```

```
docker ps
```

```
                    ↓
```

```
http://localhost:8080
```

```
                    ↓
                  DONE
```

⭐ Most important commands to remember

```
git clone https://github.com/nikithamoturi/OnlineShoppingApp.git
```

```
mvn clean install
```

```
git checkout -b feature/product-update
```

```
git diff
```

```
git commit -m "Message"
```

```
git fetch origin
```

```
git merge feature/product-update
```

```
docker build -t OnlineShopping .
```

```
docker run -d -p 8080:8080 OnlineShopping
```

```
docker push your_username/OnlineShopping:v1
```

SET-5

PART 1 — Clone and Build the Blood Bank Management System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir BloodBankLab
```

```
cd BloodBankLab
```

Now clone the repository given in the question:

```
git clone https://github.com/sarasrija/Blood-Bank-Management-system.git
```

```
cd Blood-Bank-Management-system
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins. Ensure MySQL dependency and Tomcat plugin configurations are correct as per the question requirements.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Local setup and verification (Q1, Q2)

```
git clone https://github.com/sarasrija/Blood-Bank-Management-system.git
```

```
git remote -v
```

Step 14: Check Status (Q3)

```
git status
```

Step 15: Create branch and move context (Q4)

```
git checkout -b feature/donor-registration
```

Step 16: Identify branches (Q5)

```
git branch -a
```

Step 17: Stage and commit specific file (Q6)

```
git add src/main/java/com/bloodbank/servlet/DonorServlet.java
```

```
git commit -m "Add donor registration servlet"
```

Step 18: Amend commit with missing file (Q7)

```
git add src/main/webapp/WEB-INF/web.xml
```

```
git commit --amend --no-edit
```

Step 19: Retrieve updates without modifying local (Q8)

```
git fetch origin
```

Step 20: Incorporate updates from main (Q9)

```
git merge origin/main
```

Step 21: Rebase feature branch on main (Q10)

```
git rebase origin/main
```

Step 22: Cancel rebase on conflict (Q11)

```
git rebase --abort
```

Step 23: Undo faulty commit (Q12)

```
git revert <commit-hash-of-faulty-commit>
```

Step 24: Soft reset (Q13)

```
git reset --soft HEAD~1
```

Step 25: Remove accidentally committed file from tracking (Q14)

```
git rm --cached src/main/resources/db-config.env
```

Step 26: Stash changes (Q15)

```
git stash
```

Step 27: View and restore stash (Q16)

```
git stash list
```

```
git stash pop
```

Step 28: Inspect differences (Q17)

```
git diff main..feature/donor-registration -- src/main/webapp/index.jsp
```

Step 29: Visual timeline log (Q18)

```
git log --graph --oneline --all
```

Step 30: Merge feature into main (Q19)

```
git checkout main
```

```
git merge feature/donor-registration
```

Step 31: Push main branch and verify (Q20)

```
git push origin main
```

```
git log origin/main..main
```

PART 3 — DOCKER Step 32: First find your WAR/JAR Look inside `target/` for your generated `.war` or `.jar` file.

Step 33: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM openjdk:17-jdk-slim
```

WORKDIR /app

```
COPY target/*.war /app/Blood-managent.war
```

```
CMD ["java", "-jar", "Blood-managent.war"]
```

Step 34: Build Docker image

```
docker build -t Bloodbankapp-image .
```

Step 35: Run the container

```
docker run -d -p 8080:8080 --name Bloodbank-app-container Bloodbankapp-image
```

Step 36: Check the container

```
docker ps
```

```
docker ps -a
```

```
docker exec -it Bloodbank-app-container /bin/bash
```

```
docker stop Bloodbank-app-container
```

```
docker start Bloodbank-app-container
```

Step 37: Troubleshooting and Port Mapping If it fails, check logs: `docker logs Bloodbank-app-container` Check ports: `docker port Bloodbank-app-container`

Step 38: Tag, Login, and Push

```
docker commit 0e993d2009a1 your_dockerhub_username/Bloodbankapp:v1
```

```
docker login
```

```
docker push your_dockerhub_username/Bloodbankapp:v1
```

```
docker logout
```

Step 39: Useful Docker commands to memorize

```
docker images, docker build -t [img] ., docker ps, docker ps -a, docker exec, docker commit, docker push.
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone https://github.com/sarasrija/Blood-Bank-Management-system.git
```

```
                ↓
```

```
cd Blood-Bank-Management-system
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
         dependencies
                ↓
```

```
       Maven → Update Project
                  ↓
```

```
mvn clean install
```

```
                  ↓
          BUILD SUCCESS?
             /       \
           NO         YES
           ↓           ↓
   Read Maven error   Continue
           ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                       ↓
```

```
git checkout -b feature/donor-registration
```

```
                       ↓
              Git operations
                       ↓
                 Dockerfile
                       ↓
```

```
mvn clean install
```

```
                       ↓
```

```
docker build -t Bloodbankapp-image .
```

```
                       ↓
```

```
docker run -d -p 8080:8080
```

```
          --name Bloodbank-app-container Bloodbankapp-image
                       ↓
```

```
docker ps
```

```
                       ↓
```

```
http://localhost:8080
```

```
                       ↓
                    DONE
```

⭐ Most important commands to remember

```
git clone https://github.com/sarasrija/Blood-Bank-Management-system.git
```

```
git checkout -b feature/donor-registration
```

```
git add <file>
```

```
git commit -m "msg"
```

```
git commit --amend --no-edit
```

```
git fetch origin
```

```
git rebase origin/main
```

```
git revert <commit>
```

```
git stash
```

```
docker build -t Bloodbankapp-image .
```

```
docker run -d -p 8080:8080 --name Bloodbank-app-container Bloodbankapp-image
```

```
docker commit <container-id> user/img:v1
```

```
docker push user/img:v1
```

SET-6

PART 1 — Clone and Build the Sports Management System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir SportsManagementLab
```

```
cd SportsManagementLab
```

Now clone the repository given in the question:

```
git clone https://github.com/sarasrija/SportManagementSystem.git
```

```
cd SportManagementSystem
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins. Fix any validation errors or missing JDBC dependencies.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Local setup and verification (Q1, Q2)

```
git clone https://github.com/sarasrija/SportManagementSystem.git
```

```
git remote -v
```

Step 14: Check Status (Q3)

```
git status
```

Step 15: Create branch and move context (Q4)

```
git checkout -b feature/player-registration
```

Step 16: Identify branches (Q5)

```
git branch -a
```

Step 17: Stage and commit specific file (Q6)

```
git add src/main/java/com/sports/servlet/RegistrationServlet.java
```

```
git commit -m "Add player registration servlet"
```

Step 18: Amend commit with missing file (Q7)

```
git add src/main/webapp/WEB-INF/web.xml
```

```
git commit --amend --no-edit
```

Step 19: Retrieve updates without modifying local (Q8)

```
git fetch origin
```

Step 20: Incorporate updates from main (Q9)

```
git merge origin/main
```

Step 21: Rebase feature branch on main (Q10)

```
git rebase origin/main
```

Step 22: Cancel rebase on conflict (Q11)

```
git rebase --abort
```

Step 23: Undo faulty commit (Q12)

```
git revert <commit-hash-of-faulty-commit>
```

Step 24: Soft reset (Q13)

```
git reset --soft HEAD~1
```

Step 25: Remove accidentally committed file from tracking (Q14)

```
git rm --cached src/main/resources/db-config.env
```

Step 26: Stash changes (Q15)

```
git stash
```

Step 27: View and restore stash (Q16)

```
git stash list
```

```
git stash pop
```

Step 28: Inspect differences (Q17)

```
git diff main..feature/player-registration -- src/main/webapp/index.jsp
```

Step 29: Visual timeline log (Q18)

```
git log --graph --oneline --all
```

Step 30: Merge feature into main (Q19)

```
git checkout main
```

```
git merge feature/player-registration
```

Step 31: Push main branch and verify (Q20)

```
git push origin main
```

```
git log origin/main..main
```

PART 3 — DOCKER Step 32: First find your WAR/JAR Look inside `target/` for your generated `.war` or `.jar` file.

Step 33: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM openjdk:17-jdk-slim
```

WORKDIR /app

```
COPY target/sports-management.jar /app/sports-management.jar
```

```
CMD ["java", "-jar", "sports-management.jar"]
```

Step 34: Build Docker image

```
docker build -t sportsapp-image .
```

Step 35: Run the container

```
docker run -d -p 8080:8080 --name sports-app-container sportsapp-image
```

Step 36: Check the container

```
docker ps
```

```
docker ps -a
```

```
docker exec -it sports-app-container /bin/bash
```

```
docker stop sports-app-container
```

```
docker start sports-app-container
```

Step 37: Troubleshooting and Port Mapping If it fails, check logs: `docker logs sports-app-container` Check ports: `docker port sports-app-container`

Step 38: Tag, Login, and Push

```
docker commit 0e993d2009a1 your_dockerhub_username/sportsapp:v1
```

```
docker login
```

```
docker push your_dockerhub_username/sportsapp:v1
```

```
docker logout
```

Step 39: Useful Docker commands to memorize

```
docker images, docker build -t [img] ., docker ps, docker ps -a, docker exec, docker commit, docker push.
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone https://github.com/sarasrija/SportManagementSystem.git
```

```
                ↓
```

```
cd SportManagementSystem
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
          Open pom.xml
                ↓
   Check Java version / packaging /
       dependencies
                ↓
     Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
```

```
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                       ↓
```

```
git checkout -b feature/player-registration
```

```
                       ↓
              Git operations
                       ↓
                 Dockerfile
                       ↓
```

```
mvn clean install
```

```
                       ↓
```

```
docker build -t sportsapp-image .
```

```
                       ↓
```

```
docker run -d -p 8080:8080
```

```
          --name sports-app-container sportsapp-image
                       ↓
```

```
docker ps
```

```
                       ↓
```

```
http://localhost:8080
```

```
                       ↓
                    DONE
```

⭐ Most important commands to remember

```
git clone https://github.com/sarasrija/SportManagementSystem.git
```

```
git checkout -b feature/player-registration
```

```
git add <file>
```

```
git commit -m "msg"
```

```
git commit --amend --no-edit
```

```
git fetch origin
```

```
git rebase origin/main
```

```
git revert <commit>
```

```
git stash
```

```
docker build -t sportsapp-image .
```

```
docker run -d -p 8080:8080 --name sports-app-container sportsapp-image
```

```
docker commit <container-id> user/img:v1
```

```
docker push user/img:v1
```

I'll structure this exactly in the order you should perform it in the lab, using the applications you said you have: Eclipse, Maven, Git Bash, Docker, Internet/GitHub access.

SET-7

PART 1 — Clone and Build the Apartment Management System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir ApartmentManagementWorkspace
```

```
cd ApartmentManagementWorkspace
```

Now clone the repository given in the question:

```
git clone https://github.com/sarasrija/ApartmentManagementSystem.git
```

```
cd ApartmentManagementSystem
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins. [Fixes for Set 7 based on exam questions] - Fix XML schema violation in configuration. - Add complete, corrected block for `tomcat7-maven-plugin`. - Correct the block required for MySQL connectivity. - Add the missing element in the block to fix `test-compile`.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Set up this codebase on your local machine.

```
git clone https://github.com/sarasrija/ApartmentManagementSystem.git
```

Step 14: Verify the remote server connections configured for fetching and pushing code.

```
git remote -v
```

Step 15: Inspect the current status of all staged, unstaged, and untracked changes.

```
git status
```

Step 16: Move your workspace directly into a new working context named feature/tenant-registration using a single operation.

```
git checkout -b feature/tenant-registration
```

Step 17: Determine all local and remote branches in the repository while confirming which branch you are actively on.

```
git branch -a
```

Step 18: Save specific file to the repository history with the log note "Add tenant registration servlet".

```
git add src/main/java/com/apartment/servlet/TenantServlet.java
```

```
git commit -m "Add tenant registration servlet"
```

Step 19: Merge this file into that last snapshot without altering the original message.

```
git add src/main/webapp/WEB-INF/web.xml
```

```
git commit --amend --no-edit
```

Step 20: Retrieve remote updates without modifying your local working files.

```
git fetch origin
```

Step 21: Incorporate the latest updates from the remote main branch into your currently active branch.

```
git merge origin/main
```

Step 22: Reorder your local feature branch history so that it stems from the tip of the updated main branch.

```
git rebase origin/main
```

Step 23: Cancel the rebase entirely and return your repository to its state prior to the rebase attempt.

```
git rebase --abort
```

Step 24: Safely undo the impact of that commit while preserving the commit log history.

```
git revert <commit-hash>
```

Step 25: Undo your last commit execution while ensuring all modified changes remain staged in index memory.

```
git reset --soft HEAD~1
```

Step 26: Remove this file from project version control while keeping the file saved on your local drive.

```
git rm --cached src/main/resources/db-config.env
```

Step 27: Safely store your uncommitted changes without creating a commit record.

```
git stash
```

Step 28: View all stored work snapshots and restore your saved modifications back to your workspace.

```
git stash list
```

```
git stash pop
```

Step 29: Inspect the line-by-line differences between feature/tenant-registration and main specifically for index.jsp.

```
git diff main..feature/tenant-registration -- src/main/webapp/index.jsp
```

Step 30: Produce a compact, single-line visual timeline mapping out how project branches have diverged and merged over time.

```
git log --graph --oneline --all
```

Step 31: Return to main and bring the feature branch changes into main.

```
git checkout main
```

```
git merge feature/tenant-registration
```

Step 32: Upload your updated main branch to GitHub, and verify that your local workspace and the remote repository share the exact same commit point.

```
git push origin main
```

```
git log --oneline
```

PART 3 — DOCKER Step 33: First find your WAR/JAR Look inside `target/` for your generated `.jar` or `.war` file (e.g., `apartment-management.jar`).

Step 34: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM openjdk:17-jdk-slim
```

WORKDIR /app

```
COPY target/apartment-management.jar /app/apartment-management.jar
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "apartment-management.jar"]
```

Step 35: Build Docker image

```
docker build -t apartmentapp-image .
```

Step 36: Run the container

```
docker run -d -p 8080:8080 --name apartment-app-container apartmentapp-image
```

Step 37: Check the container

```
docker ps
```

Open browser: http://localhost:8080

Step 38: Troubleshooting and Port Mapping If it fails, check logs: `docker logs apartment-app-container` Check ports: `docker port apartment-app-container`

Step 39: Useful Docker commands to memorize

```
docker images, docker build -t [img] ., docker ps, docker ps -a, docker logs, docker stop, docker rm.
```

To save container state and push to DockerHub:

```
docker commit 0e993d2009a1 your_dockerhub_username/apartmentapp:v1
```

```
docker login
```

```
docker push your_dockerhub_username/apartmentapp:v1
```

```
docker logout
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone https://github.com/sarasrija/ApartmentManagementSystem.git
```

```
                ↓
```

```
cd ApartmentManagementSystem
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
         dependencies
                ↓
      Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                      ↓
```

```
git branch
```

```
                    ↓
            Git operations
                    ↓
               Dockerfile
                    ↓
```

```
mvn clean install
```

```
                    ↓
```

```
docker build -t apartmentapp-image .
```

```
                    ↓
```

```
docker run -d -p
```

```
          8080:8080 apartmentapp-image
                    ↓
```

```
docker ps
```

```
                    ↓
```

```
http://localhost:8080
```

```
                    ↓
                  DONE
```

⭐ Most important commands to remember

```
git clone [url]
```

```
git checkout -b [branch]
```

```
git add [file]
```

```
git commit -m "[msg]"
```

```
git status
```

```
git log --graph --oneline --all
```

```
git rebase [branch]
```

```
git merge [branch]
```

```
git push origin [branch]
```

```
mvn clean install
```

```
docker build -t [name] .
```

```
docker run -d -p 8080:8080 [image]
```

```
docker ps
```

```
docker push [image]
```

I'll structure this exactly in the order you should perform it in the lab, using the applications you said you have: Eclipse, Maven, Git Bash, Docker, Internet/GitHub access.

SET-8

PART 1 — Clone and Build the Vehicle Rental Management System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir VehicleRentalWorkspace
```

```
cd VehicleRentalWorkspace
```

Now clone the repository given in the question:

```
git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
```

```
cd VehicleRentalManagementSystem
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Verify working tree and remote repository.

```
git status
```

```
git remote -v
```

Step 14: Create a suitable feature branch for updating the vehicle information page and verify it.

```
git checkout -b feature/vehicle-update
```

```
git branch
```

Step 15: Modify the vehicle.jsp page and use git diff to verify the changes.

```
git diff vehicle.jsp
```

Step 16: Stage the modified file, commit the changes with a meaningful commit message, and verify.

```
git add vehicle.jsp
```

```
git commit -m "Update vehicle type and status"
```

```
git log -1
```

Step 17: Make a small additional modification to vehicle.jsp and verify using git diff.

```
git diff vehicle.jsp
```

Step 18: Retrieve the latest changes from the remote GitHub repository without overwriting local commits.

```
git pull --rebase origin main
```

(Or run git fetch origin followed by git rebase origin/main)

Step 19: Compare the feature branch with the main branch and identify differences.

```
git diff main..feature/vehicle-update
```

Step 20: Merge the completed vehicle-page changes into the main branch, push, and verify.

```
git checkout main
```

```
git merge feature/vehicle-update
```

```
git push origin main
```

PART 3 — DOCKER Step 21: First find your WAR/JAR Look inside `target/` for your generated `.war` file (e.g. `VehicleRentalManagementSystem.war`).

Step 22: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM tomcat:9-jdk17
```

```
COPY target/VehicleRentalManagementSystem.war /usr/local/tomcat/webapps/ROOT.war
```

```
EXPOSE 8080
```

```
CMD ["catalina.sh", "run"]
```

Step 23: Build Docker image

```
docker build -t vehiclerentalapp .
```

Step 24: Run the container

```
docker run -d -p 8080:8080 --name vehiclerental-container vehiclerentalapp
```

Step 25: Check the container

```
docker ps
```

Open browser: http://localhost:8080

Step 26: Troubleshooting and Port Mapping If it fails, check logs: `docker logs vehiclerental-container` Check ports: `docker port vehiclerental-container`

Step 27: Useful Docker commands to memorize

```
docker images, docker build -t [img] ., docker ps, docker ps -a, docker logs, docker stop, docker rm.
```

To tag and push to DockerHub:

```
docker tag vehiclerentalapp your_username/vehiclerentalapp
```

```
docker login
```

```
docker push your_username/vehiclerentalapp
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
```

```
                ↓
```

```
cd VehicleRentalManagementSystem
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
         dependencies
                ↓
      Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                       ↓
```

```
git branch
```

```
                       ↓
              Git operations
                       ↓
                 Dockerfile
                       ↓
```

```
mvn clean install
```

```
                    ↓
```

```
docker build -t vehiclerentalapp .
```

```
                    ↓
```

```
docker run -d -p
```

```
          8080:8080 vehiclerentalapp
                    ↓
```

```
docker ps
```

```
                    ↓
```

```
http://localhost:8080
```

```
                    ↓
                  DONE
```

⭐ Most important commands to remember

```
git clone [url]
```

```
git status
```

```
git remote -v
```

```
git checkout -b [branch]
```

```
git diff
```

```
git add [file]
```

```
git commit -m "[msg]"
```

```
git pull --rebase origin main
```

```
git merge [branch]
```

```
git push origin [branch]
```

```
mvn clean package
```

```
docker build -t [name] .
```

```
docker run -d -p 8080:8080 [image]
```

```
docker ps
```

```
docker push [image]
```

I'll structure this exactly in the order you should perform it in the lab, using the applications you said you have: Eclipse, Maven, Git Bash, Docker, Internet/GitHub access.

SET-9

PART 1 — Clone and Build the Gym Membership Management System Project Step 1: Open Git Bash First create a folder where you want the project.

```
cd Desktop
```

```
mkdir GymManagementWorkspace
```

```
cd GymManagementWorkspace
```

Now clone the repository given in the question:

```
git clone https://github.com/nikithamoturi/GymManagementSystem.git
```

```
cd GymManagementSystem
```

Check that the project is there:

```
ls
```

You should look for: pom.xml, src.

Step 2: Check Java and Maven BEFORE opening Eclipse In Git Bash run:

```
java -version
```

```
mvn -version
```

Write down the Java version shown. This is important to fix Java compiler configuration in pom.xml.

Step 3: Open the project in Eclipse Open Eclipse > File → Import > Maven → Existing Maven Projects > Next. Choose your project folder. Select pom.xml. Finish.

Step 4: Wait for Maven dependencies Wait for "Building workspace" or downloading dependencies to finish.

Step 5: Check pom.xml Open pom.xml. Look at , Java/compiler version, , and plugins.

Step 6: Check the packaging If the project needs to be a web application, ensure packaging is:

```
<packaging>war</packaging>
```

Step 7: Check Java version in pom.xml Ensure compiler source and target match your installed Java version:

```
<properties>
```

```
<maven.compiler.source>17</maven.compiler.source>
```

```
<maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

Step 8: Update the Maven project in Eclipse Right-click the project > Maven → Update Project (Alt + F5) > OK.

Step 9: Build the project In Git Bash:

```
mvn clean install
```

If it fails, read the error (COMPILATION ERROR, missing dependency, etc.) and run `mvn dependency:tree` if needed to identify conflicts.

Step 10: Clean build Use `mvn clean install` to remove previous build outputs and ensure a clean build.

Step 11: Run tests (if applicable) Use `mvn test` to execute tests.

Step 12: Build without tests To skip tests, use `mvn package -DskipTests` or `mvn install -DskipTests`.

PART 2 — Git Once your Maven project is built, perform the Git operations required for this set.

Step 13: Display the current branch, repository status, and remote repository details.

```
git branch
```

```
git status
```

```
git remote -v
```

Step 14: Create a separate feature branch for modifying the membership information and make it active.

```
git checkout -b feature/membership-update
```

```
git branch
```

Step 15: Modify the membership information and use git diff to identify and explain changes.

```
git diff
```

Step 16: Stage the changes and create a meaningful commit.

```
git add .
```

```
git commit -m "Update membership info"
```

```
git log -1
```

Step 17: Demonstrate the difference between working-directory changes, staged changes, and committed changes.

```
git diff          # Working directory changes
```

```
git diff --staged  # Staged changes
```

```
git log -p         # Committed changes
```

Step 18: Retrieve the latest remote information and update your local branch while preserving your work.

```
git pull --rebase origin main
```

(Or git fetch origin followed by git merge origin/main)

Step 19: Undo the previous commit using Git without deleting the project files from the working directory.

```
git reset --soft HEAD~1
```

(Or git reset HEAD~1)

Step 20: Merge the completed feature branch into the main branch, resolve any merge conflicts, and push.

```
git checkout main
```

```
git merge feature/membership-update
```

```
git push origin main
```

```
git log --oneline
```

PART 3 — DOCKER Step 21: First find your WAR/JAR Look inside `target/` for your generated `.war` file (e.g. `GymManagementSystem.war`).

Step 22: Create Dockerfile Create a `Dockerfile` in the root of your project:

```
FROM tomcat:9-jdk17
```

```
COPY target/GymManagementSystem.war /usr/local/tomcat/webapps/ROOT.war
```

```
EXPOSE 8080
```

```
CMD ["catalina.sh", "run"]
```

Step 23: Build Docker image

```
docker build -t gymmanagementapp .
```

Step 24: Run the container

```
docker run -d -p 8080:8080 --name gym-container gymmanagementapp
```

Step 25: Check the container

```
docker ps
```

Open browser: http://localhost:8080

Step 26: Troubleshooting and Port Mapping If it fails, check logs: `docker logs gym-container` Check ports: `docker port gym-container`

Step 27: Useful Docker commands to memorize

```
docker images, docker build -t [img] ., docker ps, docker ps -a, docker logs, docker stop, docker rm.
```

To tag and push to DockerHub:

```
docker tag gymmanagementapp your_username/gymmanagementapp
```

```
docker login
```

```
docker push your_username/gymmanagementapp
```

⭐ YOUR EXACT LAB ORDER START ↓ Open Git Bash ↓

```
git clone https://github.com/nikithamoturi/GymManagementSystem.git
```

```
                ↓
```

```
cd GymManagementSystem
```

```
                ↓
```

```
java -version
```

```
mvn -version
```

```
                ↓
         Open Eclipse
                ↓
     Import Existing Maven Project
                ↓
           Open pom.xml
                ↓
   Check Java version / packaging /
         dependencies
                ↓
      Maven → Update Project
                ↓
```

```
mvn clean install
```

```
                ↓
         BUILD SUCCESS?
            /       \
          NO         YES
          ↓           ↓
   Read Maven error   Continue
          ↓           ↓
   Fix pom/dependency  Git
                       ↓
```

```
git status
```

```
                    ↓
```

```
git branch
```

```
                    ↓
             Git operations
                    ↓
                Dockerfile
                    ↓
```

```
mvn clean install
```

```
                    ↓
```

```
docker build -t gymmanagementapp .
```

```
                    ↓
```

```
docker run -d -p
```

```
          8080:8080 gymmanagementapp
                    ↓
```

```
docker ps
```

```
                    ↓
```

```
http://localhost:8080
```

```
                    ↓
                  DONE
```

⭐ Most important commands to remember

```
git clone [url]
```

```
git branch
```

```
git status
```

```
git remote -v
```

```
git checkout -b [branch]
```

```
git diff
```

```
git diff --staged
```

```
git add [file]
```

```
git commit -m "[msg]"
```

```
git log
```

```
git reset --soft HEAD~1
```

```
git pull --rebase origin main
```

```
git push origin [branch]
```

```
mvn clean install
```

```
docker build -t [name] .
```

```
docker run -d -p 8080:8080 [image]
```

```
docker push [image]
```